

Universidad Rafael Landívar
Facultad de Ingeniería
Compiladores
Inga. Diana Gutiérrez



Proyecto – Fase 3
Tabla de Símbolos y Comprobación de Tipos

González Cantoral, Andry Paolo – 2000924

Guatemala, 18 de mayo del 2026

INTRODUCCIÓN

El presente proyecto corresponde a la tercera fase del desarrollo del compilador para el lenguaje MiniLang. En las fases anteriores se implementó el analizador léxico, encargado de reconocer tokens, y el analizador sintáctico ascendente con PLY, encargado de validar la estructura del programa.

En esta fase se agregó el análisis semántico, integrando la tabla de símbolos y la comprobación de tipos. El sistema ahora permite validar variables declaradas, constantes, funciones, parámetros, retornos, operaciones entre tipos y asignaciones. Además, genera archivos de salida con el resultado del análisis sintáctico, el análisis semántico y la tabla de símbolos.

El objetivo principal de esta fase es comprobar que un programa MiniLang no solo esté correctamente escrito en términos léxicos y sintácticos, sino que también sea válido semánticamente.

OBJETIVOS

Objetivo general

Implementar la fase de análisis semántico para MiniLang, integrándola con el analizador léxico y el analizador sintáctico desarrollados previamente.

Objetivos específicos

- Generar una tabla de símbolos para registrar variables, constantes, funciones y parámetros.
- Validar que las variables utilizadas hayan sido declaradas previamente.
- Detectar identificadores duplicados dentro del mismo ámbito.
- Comprobar compatibilidad de tipos en asignaciones y operaciones.
- Validar el paso de parámetros en llamadas a funciones.
- Verificar que los retornos de funciones coincidan con el tipo declarado.
- Permitir coerción de tipos de int a float.
- Reportar errores semánticos indicando línea y columna.
- Generar archivos de salida para el análisis sintáctico, semántico y la tabla de símbolos.

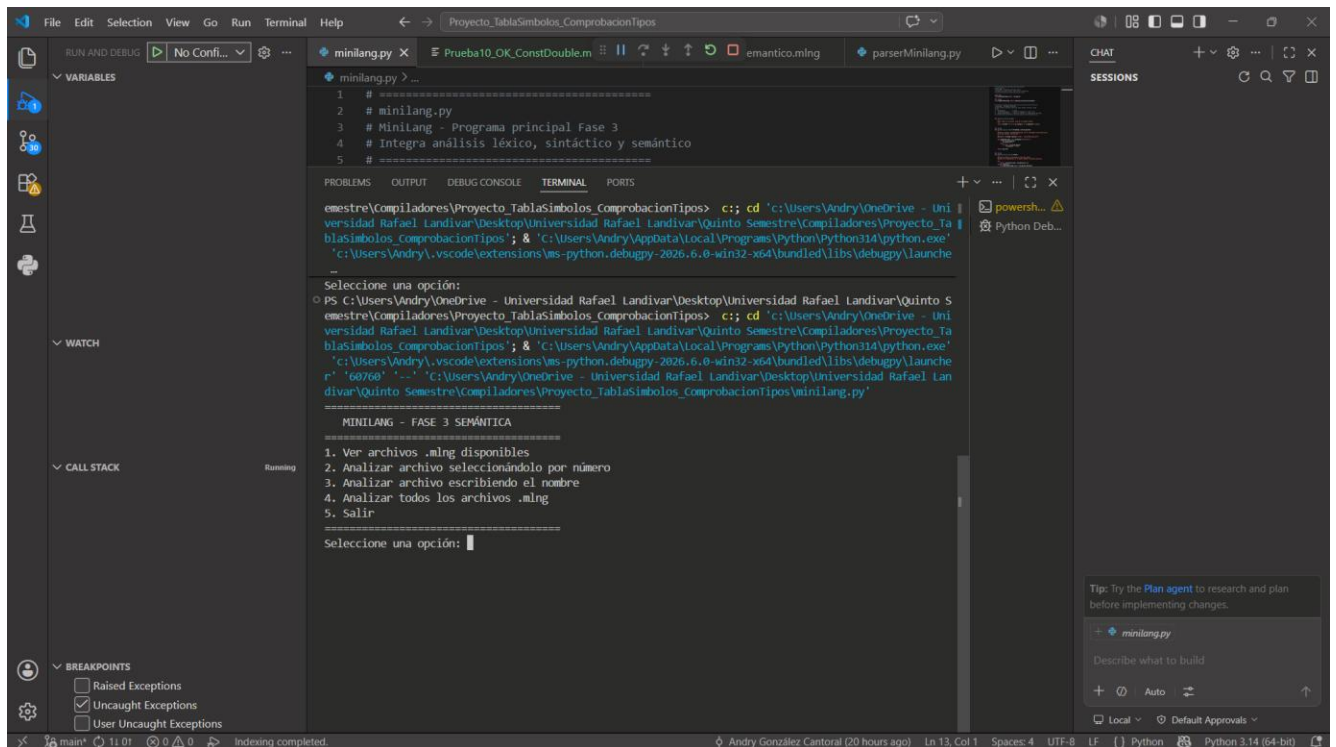
DESCRIPCIÓN DEL FUNCIONAMIENTO DEL PROGRAMA

El compilador trabaja por fases. Primero, el archivo fuente con extensión .mlng es procesado por el analizador léxico, el cual genera tokens y detecta errores léxicos. Luego, esos tokens son enviados al parser ascendente implementado con PLY, el cual valida la gramática y construye una estructura interna del programa conocida como AST.

Si el archivo no contiene errores léxicos ni sintácticos, se ejecuta el análisis semántico. En esta etapa se construye la tabla de símbolos y se revisa que las operaciones, asignaciones, funciones y condiciones sean válidas según los tipos de datos del lenguaje.

El flujo general del sistema es:

Archivo .mlng → Lexer → Parser → AST → Analizador semántico → Tabla de símbolos y resultados



```
File Edit Selection View Go Run Terminal Help
Proyecto_TablaSimbolos_ComprobacionTipos

minilang.py x Prueba10_OK_ConstDouble.m emantico.mling parserMinilang.py

1 #
2 # minilang.py
3 # Minilang - Programa principal Fase 3
4 # Integra análisis léxico, sintáctico y semántico
5 #

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

emestre\Compiladores\Proyecto_TablaSimbolos_ComprobacionTipos> c:: cd 'c:\Users\Andry\OneDrive - Uni
versidad Rafael Landivar\Desktop\Universidad Rafael Landivar\Quinto Semestre\Compiladores\Proyecto_Ta
blasimbolos_comprobacionTipos'; & 'c:\Users\Andry\AppData\Local\Programs\Python\Python314\python.exe'
'c:\Users\Andry\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launche
r'

Selecione una opción:
PS C:\Users\Andry\OneDrive - Universidad Rafael Landivar\Desktop\Universidad Rafael Landivar\Quinto S
emestre\Compiladores\Proyecto_TablaSimbolos_ComprobacionTipos> c:: cd 'c:\Users\Andry\OneDrive - Uni
versidad Rafael Landivar\Desktop\Universidad Rafael Landivar\Quinto Semestre\Compiladores\Proyecto_Ta
blasimbolos_comprobacionTipos'; & 'c:\Users\Andry\AppData\Local\Programs\Python\Python314\python.exe'
'c:\Users\Andry\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launche
r' '60760' '-.' 'c:\Users\Andry\OneDrive - Universidad Rafael Landivar\Desktop\Universidad Rafael Lan
divar\Quinto Semestre\Compiladores\Proyecto_TablaSimbolos_comprobacionTipos\minilang.py'

MINILANG - FASE 3 SEMÁNTICA

=====
1. Ver archivos .mlng disponibles
2. Analizar archivo seleccionándolo por número
3. Analizar archivo escribiendo el nombre
4. Analizar todos los archivos .mlng
5. Salir
=====
Selecione una opción: 
```

Proyecto.py

El archivo Proyecto.py corresponde al analizador léxico de MiniLang. Su función es leer el código fuente y convertirlo en una secuencia de tokens.

Para la Fase 3 se mantuvo la base del lexer de fases anteriores, pero se agregaron nuevas palabras reservadas necesarias para el análisis semántico:

- const: permite declarar constantes.
- double: se reconoce como alias de float.

El lexer también conserva el manejo de errores léxicos, como números mal formados, caracteres inesperados, cadenas sin cerrar e identificadores demasiado largos. Cada error se reporta con línea, columna, símbolo y descripción.

Gramática

Estructura general del programa

La regla principal de la gramática es program, la cual representa un programa completo de MiniLang. Esta regla permite que el archivo tenga saltos de línea opcionales al inicio o al final, y en el centro debe contener una lista de instrucciones.

program : opt_newlines stmt_list opt_newlines

La lista de instrucciones se maneja mediante stmt_list, permitiendo que un programa contenga una o varias instrucciones. Cada instrucción puede ser simple o compuesta. Las instrucciones simples incluyen declaraciones, asignaciones, llamadas a funciones, Read, Write y return. Las instrucciones compuestas incluyen if, while y definición de funciones.

Instrucciones simples

Las instrucciones simples de MiniLang son aquellas que terminan con punto y coma. La gramática reconoce las siguientes:

statement : declaration stmt_end
 | assignment stmt_end
 | write_stmt stmt_end
 | read_stmt stmt_end
 | return_stmt stmt_end
 | func_call stmt_end

El final de una instrucción se representa con stmt_end. En el lenguaje se utiliza ; para cerrar instrucciones, aunque también se conserva una recuperación con NEWLINE para que el parser pueda continuar analizando el archivo cuando falta el punto y coma. Esto permite reportar varios errores en una sola ejecución, en lugar de detenerse en el primer error.

Declaraciones de variables y constantes

La gramática permite declarar variables utilizando tipos primitivos como:

int
float
double
bool
string

En el parser, los tipos se reducen mediante la regla type. El tipo double se maneja internamente como float, ya que ambos representan valores decimales dentro del lenguaje.

Ejemplos válidos:

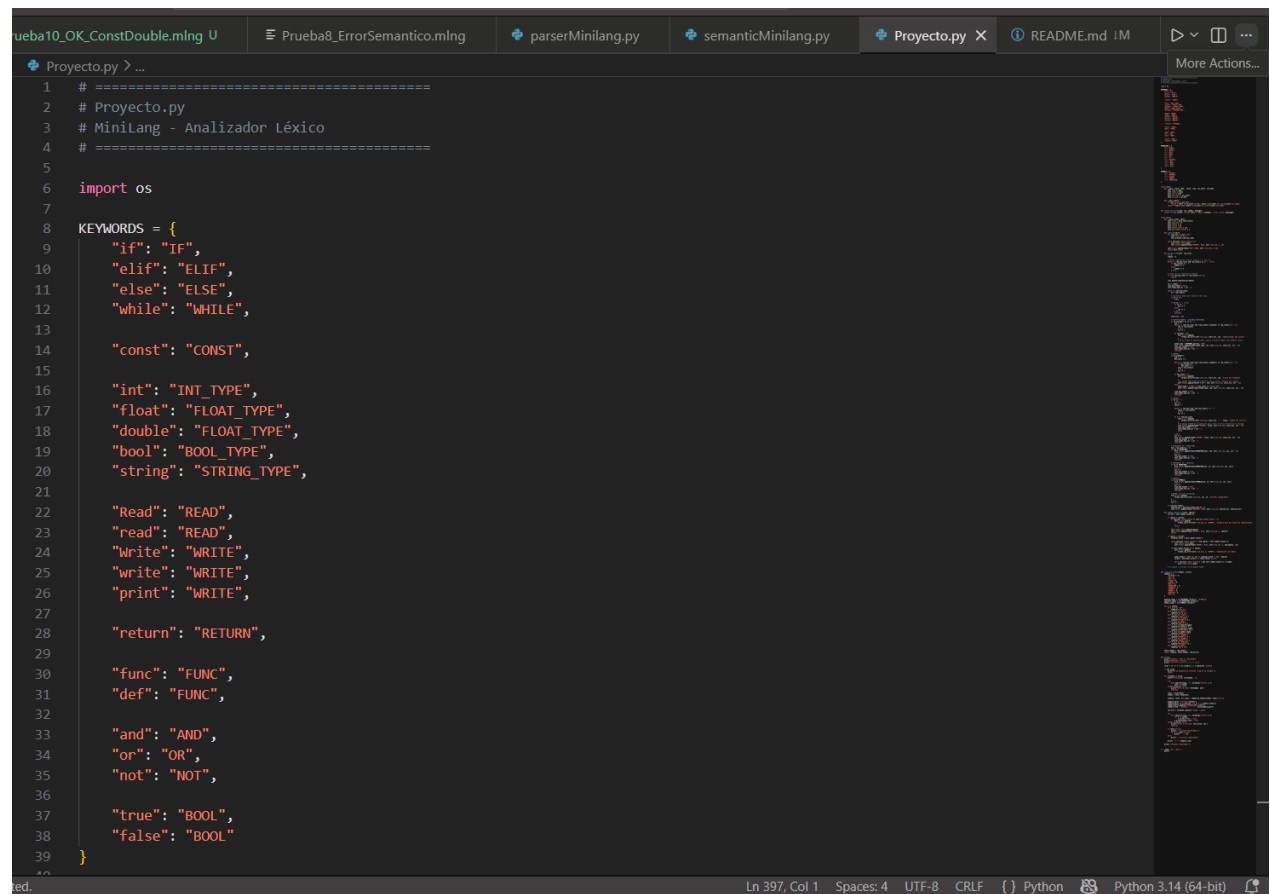
int edad;
float promedio;
double radio;
bool activo;
string nombre;

También se agregó soporte para constantes mediante const, lo cual permite declarar símbolos cuyo valor no puede ser reasignado después de haber recibido un valor.

Ejemplo:

const int limite;
limite = 100;

Esta parte es importante para la Fase 3, ya que el enunciado solicita manejar variables, constantes, asignación de valores y almacenamiento en la tabla de símbolos.



```
1 # =====
2 # Proyecto.py
3 # Minilang - Analizador Léxico
4 # =====
5
6 import os
7
8 KEYWORDS = {
9     "if": "IF",
10    "elif": "ELIF",
11    "else": "ELSE",
12    "while": "WHILE",
13
14    "const": "CONST",
15
16    "int": "INT_TYPE",
17    "float": "FLOAT_TYPE",
18    "double": "FLOAT_TYPE",
19    "bool": "BOOL_TYPE",
20    "string": "STRING_TYPE",
21
22    "Read": "READ",
23    "read": "READ",
24    "Write": "WRITE",
25    "write": "WRITE",
26    "print": "WRITE",
27
28    "return": "RETURN",
29
30    "func": "FUNC",
31    "def": "FUNC",
32
33    "and": "AND",
34    "or": "OR",
35    "not": "NOT",
36
37    "true": "BOOL",
38    "false": "BOOL"
39 }
```

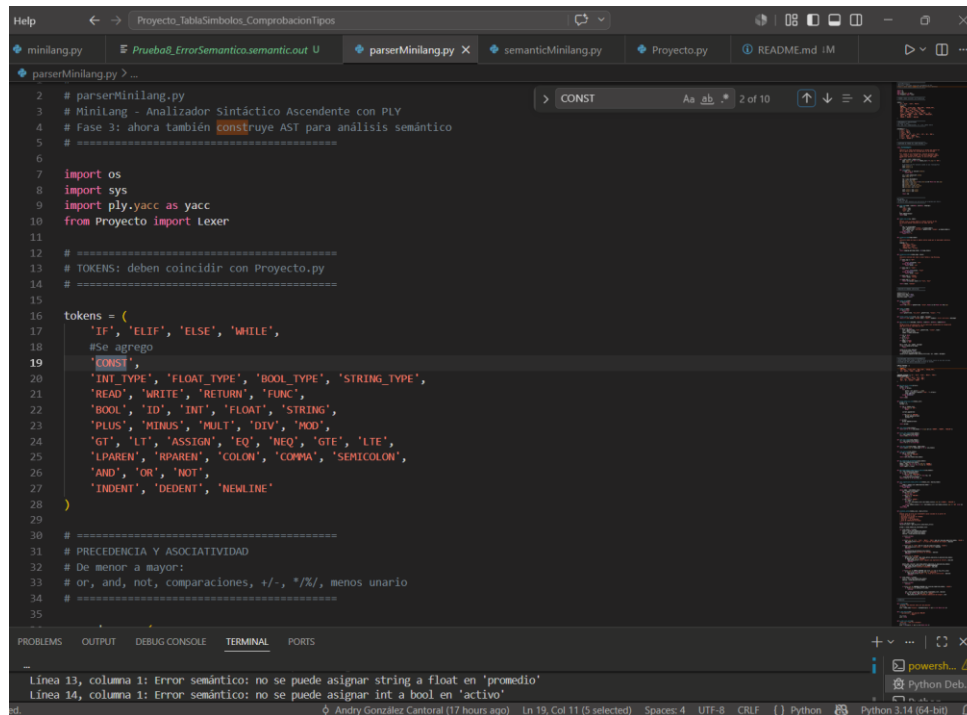
parserMinilang.py

El archivo parserMinilang.py implementa el analizador sintáctico ascendente utilizando PLY. En esta fase se mantuvo la validación de la gramática, pero se agregó la construcción del AST.

El AST es una representación interna del programa que permite al analizador semántico recorrer las instrucciones de forma ordenada. Cada construcción del lenguaje se representa como un nodo, por ejemplo:

- Program
- Declaration
- Assignment
- FunctionDef
- FunctionCall
- Return
- If
- While
- Write
- Read
- BinaryOp
- UnaryOp
- Literal
- Identifier

También se agregó soporte para declaraciones constantes mediante const y se mantuvo la recuperación de errores sintácticos para continuar el análisis hasta EOF.



```
2 # parserMinilang.py
3 # Minilang - Analizador Sintáctico Ascendente con PLY
4 # Fase 3: ahora también construye AST para análisis semántico
5 # =====
6
7 import os
8 import sys
9 import ply.yacc as yacc
10 from Proyecto import Lexer
11
12 # =====
13 # TOKENS: deben coincidir con Proyecto.py
14 # =====
15
16 tokens = (
17     'IF', 'ELIF', 'ELSE', 'WHILE',
18     #Se agrego
19     'CONST',
20     'INT_TYPE', 'FLOAT_TYPE', 'BOOL_TYPE', 'STRING_TYPE',
21     'READ', 'WRITE', 'RETURN', 'FUNC',
22     'BOOL', 'ID', 'INT', 'FLOAT', 'STRING',
23     'PLUS', 'MINUS', 'MULT', 'DIV', 'MOD',
24     'GT', 'LT', 'ASSIGN', 'EQ', 'NEQ', 'GTE', 'LTE',
25     'LPAREN', 'RPAREN', 'COLON', 'COMMA', 'SEMICOLON',
26     'AND', 'OR', 'NOT',
27     'INDENT', 'DEDENT', 'NEWLINE'
28 )
29
30 # =====
31 # PRECEDENCIA Y ASOCIATIVIDAD
32 # De menor a mayor:
33 # or, and, not, comparaciones, +/-, */%, menos unario
34 # =====
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Linea 13, columna 1: Error semántico: no se puede asignar string a float en 'promedio'

Linea 14, columna 1: Error semántico: no se puede asignar int a bool en 'activo'

```
50 class PlyTokenAdapter:
51     def __init__(self, p):
52         self.p = p
53         self.tokens = []
54         self.index = 0
55         return tok
56
57
58 # =====
59 #Se agrego
60 # NODOS PARA AST
61 # El parser ahora devuelve una estructura del programa para Fase 3.
62 # =====
63
64 def make_node(node, line=None, col=None, **kwargs):
65     data = {
66         "node": node,
67         "line": line,
68         "col": col,
69     }
70     data.update(kwargs)
71     return data
72
73
74 def token_line_col(p, index):
75     """
76     Obtiene línea y columna desde un símbolo terminal de PLY.
77     Se usa para guardar ubicación en los nodos del AST.
78     """
79     try:
80         tok = p.slice[index]
81         line = getattr(tok, "lineno", p.lineno(index))
82         col = getattr(tok, "col_start", getattr(tok, "lexpos", p.lexpos(index)))
83         return line, col
84     except Exception:
85         return "?", "?"
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
```

Linea 13, columna 1: Error semántico: no se puede asignar string a float en 'promedio'

Linea 14, columna 1: Error semántico: no se puede asignar int a bool en 'activo'

semanticMinilang.py

El archivo semanticMinilang.py fue agregado en la Fase 3 y contiene el análisis semántico del compilador.

Este archivo se encarga de recorrer el AST generado por el parser, construir la tabla de símbolos y validar las reglas semánticas del lenguaje. Entre sus principales responsabilidades están:

- Registrar variables, constantes, funciones y parámetros.
- Validar variables no declaradas.
- Detectar identificadores duplicados.
- Validar asignaciones entre tipos compatibles.
- Comprobar operaciones aritméticas, relacionales y lógicas.
- Validar condiciones de if y while.
- Verificar llamadas a funciones.
- Validar cantidad y tipo de argumentos.
- Comprobar retornos de funciones.
- Detectar reasignación de constantes.

La tabla de símbolos almacena información como nombre, categoría, tipo, valor, ámbito, línea y columna.

Help ← → Proyecto_TablaSimbolos_ComprobacionTipos

minilang.py Prueba8_ErrorSemantico.semantic.out U parserMinilang.py semanticMinilang.py X Proyecto.py README.md IM

semanticMinilang.py > Symbol

```
7 from dataclasses import dataclass, field
8 import os
9 import sys
10
11
12 # =====
13 #Se agrego
14 # MODELOS DE TABLA DE SÍMBOLOS
15 # =====
16
17 #Se agrego
18 @dataclass
19 class Symbol:
20     name: str
21     category: str
22     data_type: str
23     scope: str
24     line: int | str | None = None
25     col: int | str | None = None
26     value: object = None
27     is_const: bool = False
28     assigned: bool = False
29     params: list = field(default_factory=list)
30     return_type: str | None = None
31
32
33 #Se agrego
34 class SymbolTable:
35     """
36     Maneja símbolos por ámbitos.
37     Permite insertar variables, constantes, funciones y parámetros.
38     """
39
40     def __init__(self):
```

class symbol Aa ab .* 1 of 2

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

...
Linea 13, columna 1: Error semántico: no se puede asignar string a float en 'promedio'
Linea 14, columna 1: Error semántico: no se puede asignar int a bool en 'activo'

Andry González Cantoral (7 hours ago) Ln 19, Col 13 (12 selected) Spaces: 4 UTF-8 LF Python Python 3.14 (64-bit)

Minilang.py

El archivo minilang.py funciona como programa principal del compilador. Contiene un menú que permite seleccionar archivos .mlng para analizarlos individualmente o ejecutar todos los archivos de prueba.

En Fase 3, este archivo fue modificado para integrar el análisis semántico. Ahora el flujo de ejecución es:

1. Lee el archivo .mlng.
2. Ejecuta el lexer y parser.
3. Si existen errores léxicos o sintácticos, no ejecuta semántica.
4. Si el archivo pasa lexer y parser, ejecuta el análisis semántico.
5. Genera los archivos de salida correspondientes.

Los archivos generados son:

- .parserMinilang.out
- .semantic.out
- .symbols.out

```
var desktop (C:\Users\Andr\Documents\Quinto Semestre (compiladores)\Proyecto_Tabla de Símbolos y Comprobación de Tipos (minilang.py)
=====
MINILANG - FASE 3 SEMÁNTICA
=====
1. Ver archivos .mlng disponibles
2. Analizar archivo seleccionándolo por número
3. Analizar archivo escribiendo el nombre
4. Analizar todos los archivos .mlng
5. Salir
=====
Seleccione una opción: [
```

Tabla de Símbolos y Comprobación de Tipos

La tabla de símbolos permite almacenar y consultar los identificadores encontrados durante el análisis del programa. Cada símbolo registrado contiene información necesaria para la comprobación semántica.

Los campos utilizados son:

- Nombre: identificador del símbolo.
- Categoría: variable, constante, función o parámetro.
- Tipo: int, float, string o bool.
- Valor: valor calculado cuando es posible.
- Ámbito: contexto donde existe el símbolo, por ejemplo global o func:sumar.
- Línea y columna: ubicación de declaración.

La tabla permite diferenciar variables globales, parámetros y variables locales dentro de funciones. Esto evita conflictos entre símbolos de distintos ámbitos.

1	TABLA DE SÍMBOLOS - MINILANG							
2	=====							
3	Nombre	Categoría	Tipo	Valor	Ámbito	Línea	Columna	Parámetros
4	-----							
5	a	constante	int	70	global	3	11	-
6	radio	variable	float	20.5	global	6	8	-
7	circunferencia	variable	float	128.8056	global	9	8	-
8								

Pruebas Realizadas

SEARCH
PROYECTO TABLASIMBOLOS_COMPROBACION TIPOS
Prueba1_OK_HolaMundo.mInq
Prueba1_OK_HolaMundo.parserMinilan... U
Prueba1_OK_HolaMundo.semantic.out U
Prueba1_OK_HolaMundo.symbols.out U
Prueba2_OK_Aritmetica.mInq
Prueba2_OK_Aritmetica.parserMinilang.... U
Prueba2_OK_Aritmetica.semantic.out U
Prueba2_OK_Aritmetica.symbols.out U
Prueba3_OK_InputIf.mInq
Prueba3_OK_InputIf.parserMinilang.out U
Prueba3_OK_InputIf.semantic.out U
Prueba3_OK_InputIf.symbols.out U
Prueba4_OK_FuncionParametros.mInq
Prueba4_OK_FuncionParametros.parser... U
Prueba4_OK_FuncionParametros.semant... U
Prueba4_OK_FuncionParametros.symbol... U
Prueba5_OK_WhileEntradaSalida.mInq
Prueba5_OK_WhileEntradaSalida.parser... U
Prueba5_OK_WhileEntradaSalida.seman... U
Prueba5_OK_WhileEntradaSalida.symbol... U
Prueba6_ErrorLexico.mInq
Prueba6_ErrorLexico.parserMinilang.out U
Prueba7_ErrorSintactico.mInq
Prueba7_ErrorSintactico.parserMinilang.... U
Prueba8_ErrorSemantico.mInq
Prueba8_ErrorSemantico.parserMinilang... U
Prueba8_ErrorSemantico.semantic.out U
Prueba8_ErrorSemantico.symbols.out U
Prueba9_OK_TodosCasosGrande.mInq
Prueba9 OK TodosCasosGrande.parser... U
OUTLINE
TIMELINE